Interfejs a reprezentacja

Umożliwienie zmiany reprezentacji

W dobrze zaprojektowanej klasie użytkownik (programista używający klasy) powinien mieć dostęp do funkcjonalności obiektu, ale nie musi znać szczegółów jego wewnętrznej reprezentacji. Dzięki enkapsulacji możemy w przyszłości zmienić sposób przechowywania danych w klasie (np. zamiast promienia koła przechowywać jego obwód), bez konieczności modyfikowania kodu, który z tej klasy korzysta. To pozwala na modyfikację programu (rozwój, optymalizację) bez ryzyka „zepsucia" istniejącej funkcjonalności. Enkapsulacja chroni więc nie tylko dane, ale także interfejs klasy — czyli sposób, w jaki użytkownik jej używa.

Jako przykład użyjemy klasy `Kolo` reprezentującej koło o zadanym promieniu.

In [2]:
```
import math


class Kolo:
    def __init__(self, promien):
        self.promien = promien

    def obwod(self):
        return 2 * math.pi * self.promien

    def pole(self):
        return math.pi * self.promien**2
```

Koło ma trzy składowe — pole `promien` i metody `obwod` i `pole`. Możemy za ich pomocą odczytać odpowiednie wartości. Dodatkowo za pomocą podstawienia pod pole `promien` możemy zmienić promień koła.

Funkcja `przetestuj_kolo` pokazuje przykładowe użycie klasy `Kolo`:

In [3]:
```
def przetestuj_kolo():
    k = Kolo(1.0)
    print(f"promień: {k.promien}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
    k.promien = 10.0
    print(f"promień: {k.promien}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
```

In [4]:
```
przetestuj_kolo()
```

```
promień: 1.0
pole: 3.141592653589793
obwód: 6.283185307179586
promień: 10.0
pole: 314.1592653589793
obwód: 62.83185307179586
```

W tej wersji nie używamy enkapsulacji — pole promień jest publiczne i możemy zmienić promień koła zmieniając wartość tego pola. Tak jak w przypadku klasy `Konto` bez użycia enkapsulacji tak i tutaj nic nas nie chroni przed „popsuciem” obiektu typu `Kolo`:

In [5]:
```
k = Kolo(5)
```

In [6]:
```
k.promien = -5
```

In [7]:
```
k.obwod()
```

Out[7]:
```
-31.41592653589793
```

Dodatkowo nie jest teraz łatwa zmiana reprezentacji. Jeżeli koło reprezentujemy jako promień, to obwód musi być wyliczany. Jeżeli stwierdzimy za jakiś czas, że lepiej byłoby przechowywać obwód jako pole, a wyliczać promień, to możemy zmienić klasę w następujący sposób:

In [8]:
```
import math


class Kolo:
    def __init__(self, promien):
        self.obwod = 2 * math.pi * promien

    def promien(self):
        return self.obwod / (2 * math.pi)

    def zmien_promien(self, promien):
        self.obwod = 2 * math.pi * promien

    def pole(self):
        return self.obwod**2 / (4 * math.pi)
```

Próbujemy teraz użyć funkcji `przetestuj_kolo`. Funkcja nadal próbuje wywołać składową `obwod` jak metodę, choć po zmianie reprezentacji składowa ta zawiera liczbę. Wywołanie zakończy się zatem wyjątkiem `TypeError`:

In [9]:
```
przetestuj_kolo()
```

```
promień: <bound method Kolo.promien of <__main__.Kolo object at 0x7d804979df
d0>>
pole: 3.141592653589793
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 przetestuj_kolo()

Cell In[3], line 5, in przetestuj_kolo()
      1 def przetestuj_kolo():
      2     k = Kolo(1.0)
      3     print(f"promień: {k.promien}")
      4     print(f"pole: {k.pole()}")
----> 5     print(f"obwód: {k.obwod()}")
      6     k.promien = 10.0
      7     print(f"promień: {k.promien}")
      8     print(f"pole: {k.pole()}")

TypeError: 'float' object is not callable
```

Niestety nie dało się zmienić reprezentacji koła tak, żeby kod używający klasy pozostał bez zmian. Teraz składowa `obwod` jest polem, a składowa `promien` metodą. Dodatkowo zmianę promienia musimy teraz wykonać za pomocą metody. Musimy przepisać funkcję `przetestuj_kolo`:

In [10]:
```
def przetestuj_kolo():
    k = Kolo(1.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod}")
    k.zmien_promien(10.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod}")
```

In [11]:
```
przetestuj_kolo()
```

```
promień: 1.0
pole: 3.141592653589793
obwód: 6.283185307179586
promień: 10.0
pole: 314.1592653589793
obwód: 62.83185307179586
```

Widzimy, że zmiana reprezentacji powoduje, że trzeba przepisać cały kod korzystający z tej klasy. Dzieje się tak dlatego, że w tej wersji zmiana reprezentacji spowodowała również zmianę publicznego interfejsu klasy: inaczej odczytujemy promień i obwód oraz inaczej zmieniamy promień. Kod korzystający z klasy musi dostosować się do tych zmian.

Gdybyśmy od początku używali mechanizmu enkapsulacji, to pierwsza wersja klasy `Kolo` mogłaby wyglądać tak (możemy od razu sprawdzić poprawność promienia):

In [12]:
```
import math


class Kolo:
    def __init__(self, promien):
        self._sprawdz_promien(promien)
        self._promien = promien

    def _sprawdz_promien(self, promien):
        if not isinstance(promien, (int, float)):
            raise TypeError("promień musi być liczbą")
        if promien <= 0:
            raise ValueError("promień musi być dodatni")

    def promien(self):
        return self._promien

    def zmien_promien(self, promien):
        self._sprawdz_promien(promien)
        self._promien = promien

    def obwod(self):
        return 2 * math.pi * self._promien

    def pole(self):
        return math.pi * self._promien**2
```

Funkcja `przetestuj_kolo` mogłaby wtedy wyglądać tak (nie ma w niej żadnego odwołania do pól):

In [13]:
```
def przetestuj_kolo():
    k = Kolo(1.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
    k.zmien_promien(10.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
```

In [14]:
```
przetestuj_kolo()
```

```
promień: 1.0
pole: 3.141592653589793
obwód: 6.283185307179586
promień: 10.0
pole: 314.1592653589793
obwód: 62.83185307179586
```

Jeżeli teraz chcemy zmienić reprezentację (przechowywać obwód a liczyć promień), to możemy to zrobić tak:

In [15]:
```
import math
```

```python
class Kolo:
    def __init__(self, promien):
        self._sprawdz_promien(promien)
        self._obwod = 2 * math.pi * promien

    def _sprawdz_promien(self, promien):
        if not isinstance(promien, (int, float)):
            raise TypeError("promień musi być liczbą")
        if promien <= 0:
            raise ValueError("promień musi być dodatni")

    def promien(self):
        return self._obwod / (2 * math.pi)

    def zmien_promien(self, promien):
        self._sprawdz_promien(promien)
        self._obwod = 2 * math.pi * promien

    def obwod(self):
        return self._obwod

    def pole(self):
        return self._obwod**2 / (4 * math.pi)
```

Funkcja `przetestuj_kolo` nie wymaga teraz żadnych zmian.

In [16]:
```python
def przetestuj_kolo():
    k = Kolo(1.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
    k.zmien_promien(10.0)
    print(f"promień: {k.promien()}")
    print(f"pole: {k.pole()}")
    print(f"obwód: {k.obwod()}")
```

In [17]:
```python
przetestuj_kolo()
```

```
promień: 1.0
pole: 3.141592653589793
obwód: 6.283185307179586
promień: 10.0
pole: 314.1592653589793
obwód: 62.83185307179586
```